Study Material — Part 2: Agentic AI, Agent Bricks, MCP Servers & Multi-Agent Systems

Deliverable 4 (Part 2 of several) — Databricks GenAI Engineer Associate

This covers most of Section 1's agent objectives, Section 3's agentic-development objectives, and Section 4's MCP/memory/CI-CD-for-agents objectives — the second-heaviest tested cluster, and the newest area of the exam (updated March 2026).

Topic 6: What Is an "Agent" (vs. a Chain, vs. a Simple LLM Call)?

What is it?

- A **simple LLM call**: one prompt in, one response out. No memory, no tools, no decision-making about *how* to answer.
- A **chain**: a fixed, predetermined sequence of steps (e.g., retrieve → augment → generate). The *order* of operations is decided by you, the engineer, in advance.
- An **agent**: the LLM itself decides, at runtime, what steps to take, in what order, and whether to call external tools — based on reasoning about the task. The control flow is dynamic, not hardcoded.

Why is it used?

Some problems can't be solved by a fixed sequence of steps because the right next action depends on what happened in the previous step (e.g., "check inventory, and only if it's out of stock, look up restock dates and notify the customer"). Agents let the LLM adaptively plan and act.

How does it work?

The dominant pattern is **ReAct** (Reasoning + Acting):

1. The LLM produces a **reasoning trace** ("I need to find out X before I can answer")
2. Based on that reasoning, it decides whether to call a **tool** (a function, API, database query, etc.)
3. The tool's result is fed back to the LLM
4. The LLM repeats — reason, act, observe — until it has enough information to produce a final answer

This is different from **Chain-of-Thought (CoT)** prompting, which only generates a reasoning trace in text but does **not** interleave real tool calls mid-reasoning — CoT is "think step by step," ReAct is "think, act, observe, repeat."

Important terminology

- **Tool / function calling** — giving the LLM a defined set of callable functions (with descriptions) it can invoke when it decides more information or an action is needed
- **ReAct** — the reasoning-and-acting agent pattern described above
- **Multi-stage reasoning** — breaking a complex task into ordered sub-steps, some of which may require tools
- **Compound AI system** — an umbrella term for any GenAI application built from multiple components (retrievers, LLMs, tools, other agents) working together, as opposed to a single model call

Databricks implementation

- The **Mosaic AI Agent Framework** is Databricks' toolkit (working with MLflow) for authoring, evaluating, and deploying custom agents in code (Python), typically using frameworks like LangChain/LangGraph
- The `ChatAgent` interface is Databricks' standard interface for agent authoring that plugs into MLflow logging, evaluation, and deployment, and which automatically captures **operational metrics** (request volume, latency, errors) once deployed
- **Agent Bricks** (Topic 7) is Databricks' lower-code/no-code alternative for common agent patterns

Real-world example

A travel-booking agent: user asks "book me a flight to Chicago next week and a hotel near downtown." The agent reasons: "I need flight options first" → calls a flight-search tool → observes results → reasons "now I need hotel options near downtown Chicago" → calls a hotel-search tool → observes results → synthesizes a final combined recommendation. None of this sequence was hardcoded — the agent decided the order and which tools to call based on the request.

Simple example (conceptual ReAct trace)

```
User: "What's the weather in the city where our biggest customer is
headquartered?"

Thought: I need to find out where our biggest customer is headquartered.
Action: call_tool(get_customer_info, customer="biggest")
Observation: "Acme Corp, headquartered in Denver, CO"

Thought: Now I need the weather in Denver.
Action: call_tool(get_weather, city="Denver")
Observation: "58°F, partly cloudy"

Thought: I have what I need.
Final Answer: "It's currently 58°F and partly cloudy in Denver, where Acme Corp
(your biggest customer) is headquartered."
```

Important exam points

- **ReAct vs. Chain-of-Thought:** CoT reasons without acting; ReAct interleaves reasoning with real tool calls, adapting the plan as new information arrives. If a question describes needing to "plan and adapt actions while performing complex reasoning" with external tools, that's ReAct, not CoT.
- **Defining and ordering tools for multi-stage reasoning** is a directly tested Section 1 objective — practice thinking through "what tools does this task need, and in what logical order might they be called."
- `ChatAgent`**'s automatically captured metrics** are operational only (volume, latency, errors) — quality metrics (correctness, guideline adherence) require you to configure scorers/judges explicitly.

Common mistakes/traps

- Calling any multi-step LLM workflow an "agent" — if the sequence of steps is fixed in advance by your code, it's a chain, not an agent (agents make runtime decisions about control flow).
- Assuming CoT prompting alone gives you tool-use capability — it doesn't; you need an agentic framework/pattern (like ReAct) to actually invoke external tools mid-reasoning.

Comparison with related concepts

	Simple LLM call	Chain	Agent
Control flow	None (single call)	Fixed, predetermined by engineer	Dynamic, decided by LLM at runtime
Tool use	No	Possible but in a fixed sequence	Yes, decided adaptively
Best for	Simple, one-shot tasks	Well-defined, repeatable pipelines (e.g., standard RAG)	Tasks needing adaptive multi-step reasoning

Example exam questions (style)

- "An LLM pipeline must plan and adapt its actions while performing complex, multi-stage reasoning that leverages external tools. Which approach fits?" → Implement a ReAct-style framework, not CoT alone and not a fixed sequential chain.

Topic 7: Agent Bricks

What is it?

Agent Bricks is Databricks' **enterprise, lower-code agent platform**: instead of writing all agent logic yourself in Python, you describe your business problem and connect governed data, and Databricks automatically builds, evaluates, tunes, and deploys an agent optimized for your use case.

Why is it used?

Building high-quality custom agents from scratch (writing prompts, tuning them, building evaluation harnesses, optimizing cost/quality tradeoffs) is time-consuming and requires deep expertise. Agent Bricks automates much of this using techniques from Mosaic AI Research (domain-specific synthetic data generation, task-aware benchmarks, automatic prompt/quality optimization).

How does it work?

You pick one of four agent types based on your problem shape, connect your governed (Unity Catalog) data, and Agent Bricks handles building, benchmarking, and tuning automatically. Each agent type is hosted at an HTTP endpoint your application can call once ready.

Important terminology & the four Agent Bricks types

1. **Knowledge Assistant** — turns your documents into a RAG-style chatbot with citations. Ideal for Q&A over manuals, policies, support docs. Handles retrieval, reasoning, and response generation for you, taps into Unity Catalog volumes or vector indexes.
2. **Information Extraction Agent** — converts unstructured documents (emails, PDFs, reports) into structured fields/tables (e.g., pulling names, dates, product details out of supplier PDFs).
3. **Custom LLM Agent** — for custom text-transformation tasks: summarization, classification, other text-in/text-out transformations that don't fit the other three categories.
4. **Multi-Agent Supervisor** — the **top-level orchestrator** that interprets incoming user requests and **delegates** them to the correct specialized sub-agent (which can include Knowledge Assistants, Genie Spaces, UC Functions, and tools via MCP). Improves coordination between agents and supports natural-language feedback tuning from SMEs.

Databricks implementation

- Activated from the "Agents" section under AI/ML in the workspace sidebar
- Runs natively on Unity Catalog, so agents use **governed data** with full access control and lineage — auditable/monitorable like any other production workload
- Supports continuous improvement via feedback loops, including **Agent Learning from Human Feedback (ALHF)**
- Multi-Agent Supervisor: currently supports up to 10 sub-agents per supervisor system, integrates with Genie rooms, Knowledge Assistants, AI functions, and (increasingly) MCP

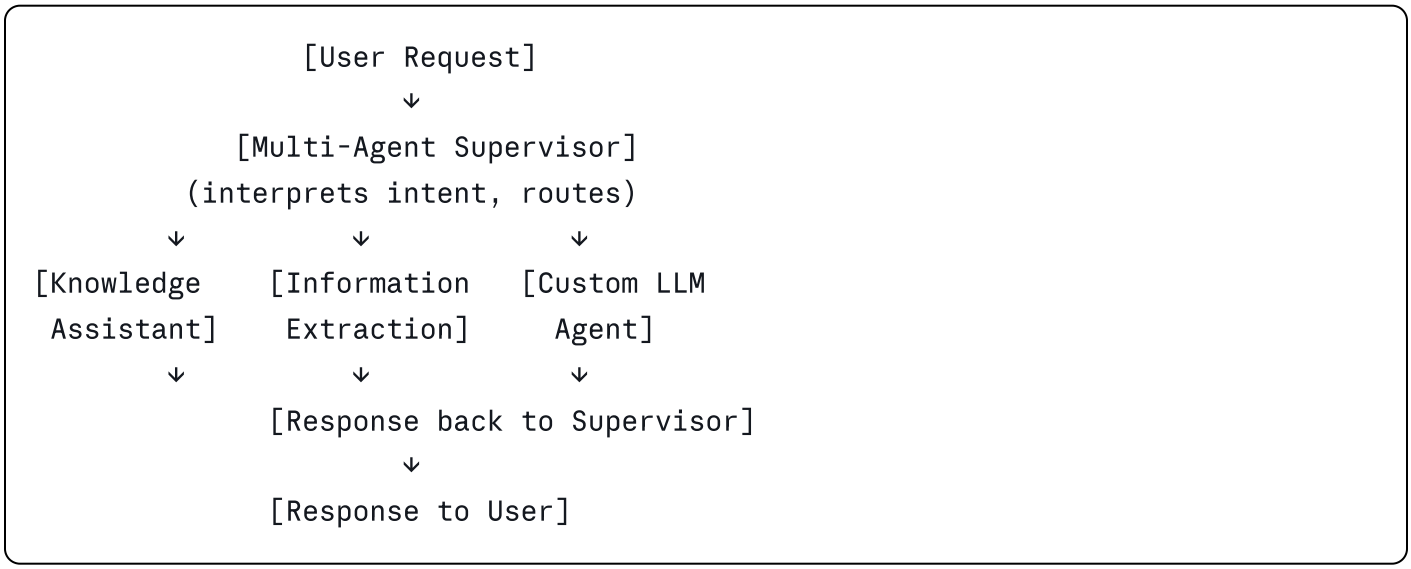
Real-world example

A manufacturing company builds: (1) a Knowledge Assistant over SOPs and maintenance manuals so technicians get cited answers instantly, (2) an Information Extraction Agent to pull structured fields from supplier PDFs, and (3) a Multi-Agent Supervisor that routes technician questions to the Knowledge Assistant and routes supplier-document processing requests to the Information Extraction Agent — all under one entry point.

Simple example

```
User asks the Supervisor: "What's the torque spec for bolt X on machine Y?"
→ Supervisor interprets intent: this is a documentation lookup question
→ Delegates to: Knowledge Assistant (trained on maintenance manuals)
→ Knowledge Assistant retrieves relevant manual section, responds with citation
→ Supervisor returns the answer to the user
```

Architecture/workflow



Important exam points

- **Know the "why" for each type**, not just the name — the exam tests reasoning about *why* an agent type fits ("because it interprets incoming requests and delegates," not just "Multi-Agent Supervisor" as a label). Two answer options can both name the correct component but only one has the correct justification (see your Q46).
- The Multi-Agent Supervisor is the correct choice **specifically when the system must serve as a top-level orchestrator across multiple distinct capabilities** — not when you just need a single RAG-style Q&A bot (that's Knowledge Assistant alone).
- Agent Bricks vs. the Agent Framework: Agent Bricks = lower-code, Databricks-managed optimization; Agent Framework = full custom code control via MLflow + `ChatAgent`. Know when each is the better fit for a described scenario (a described Section 1 objective: "determine how and when to use Agent Bricks").

Common mistakes/traps

- Choosing "Knowledge Assistant" as the top-level orchestrator when a system needs to route between *multiple distinct capabilities* (RAG, extraction, custom transformation) — Knowledge Assistant is a specialized agent type, not designed to be the routing layer.

- Assuming Agent Bricks requires you to write custom SQL or retrieval logic yourself — a large part of its value is that it automates this.

Comparison with related concepts

	Agent Bricks	**Agent Framework (custom code)**
Code required	Minimal/low-code	Full Python authoring
Optimization	Automatic (Mosaic AI Research techniques)	Manual, engineer-driven
Best for	Common patterns (Q&A, extraction, routing)	Highly custom/novel agent logic
Governance	Native Unity Catalog integration	Also integrates with UC, but you wire it yourself

Example exam questions (style)

- "A compound AI system must answer questions from guideline documents, extract event codes from reports, and route ambiguous requests to the right specialized handler, all through one entry point. Which Agent Brick serves as the top-level orchestrator?" → Multi-Agent Supervisor, because it interprets incoming requests and delegates to specialized handlers.

Topic 8: Model Context Protocol (MCP) Servers

What is it?

MCP (Model Context Protocol) is a standard way for an agent to discover and call external "tools" (data sources, APIs, actions) through a consistent interface, regardless of who built or hosts the underlying tool. An "MCP server" exposes one or more tools to an agent via this protocol.

Why is it used?

Before MCP-style standards, every tool integration required custom, one-off code. MCP lets agents plug into a growing ecosystem of standardized tool servers — some maintained by Databricks, some by third parties, some you build yourself — without reinventing the integration each time.

How does it work?

There are three categories of MCP servers, and the exam directly tests **when to use which**:

1. **Managed MCP servers** — provided and hosted by Databricks for common resources (e.g., **Unity Catalog** access). Lowest effort, built-in governance, no infrastructure to maintain.
2. **External MCP servers** — provided by a third party for their own service/API (e.g., Slack, a search API). You configure connection details and store credentials (e.g., in **Databricks Secrets**), but you don't build or host the server yourself.
3. **Custom MCP servers** — built by you, for logic/data that is genuinely proprietary to your organization and has no existing managed or external server (e.g., a proprietary risk-scoring algorithm).

Important terminology

- **Tool** — a single callable capability exposed by an MCP server
- **Databricks Secrets** — the secure storage mechanism for credentials (like API keys) an external MCP server integration needs
- **Unity Catalog MCP server** — the specific managed server Databricks provides for governed access to UC tables/functions

Databricks implementation

- Configure a managed MCP server by specifying the server type as "managed" plus the relevant resource identifier
- Deploy/connect an external MCP server by providing its connection details and referencing stored credentials (Secrets) in the MCP server configuration
- Build a custom MCP server only for the piece of functionality that is genuinely unique to your organization — everything else should use managed or external servers

Real-world example

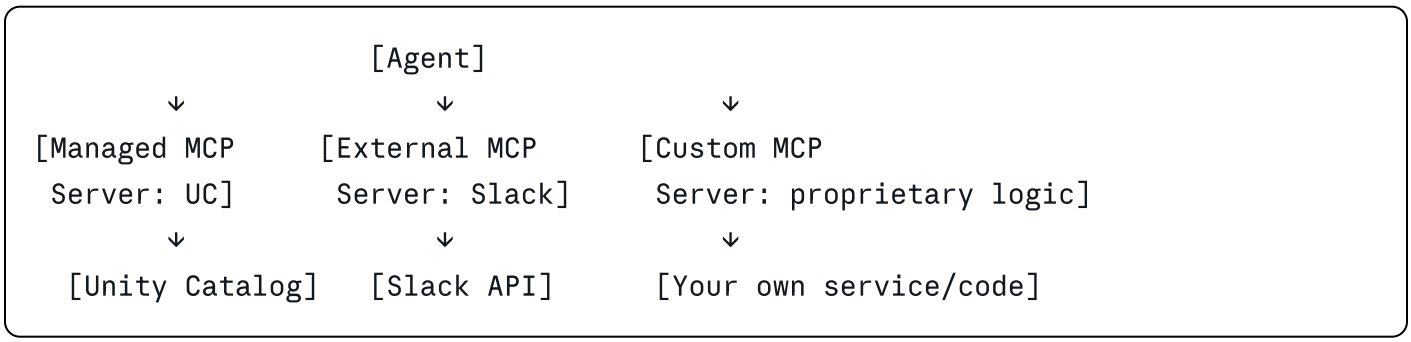
A research assistant agent needs: (1) access to a Databricks-managed knowledge/data resource, and (2) an external web-search API requiring an API key. The correct integration: configure the managed server via its identifier, and separately deploy the external server with its API key stored in Databricks Secrets — **not** a single custom server wrapping both (unnecessary reinvention) and **not** a Unity Catalog "cache" of the external content (doesn't meet a "reliable access in production" requirement well and adds unneeded complexity).

Simple example

```
Requirement: agent needs (1) Unity Catalog table access, (2) Slack
notifications,
             (3) a proprietary risk-scoring calculation

Correct integration:
1. Unity Catalog → use Databricks' MANAGED MCP server (no custom work needed)
2. Slack → use the available EXTERNAL MCP server (configure connection + auth)
3. Risk scoring → build a CUSTOM MCP server (this logic is unique to the org)
```

Architecture/workflow



Important exam points

- **Default rule of thumb, directly tested:** use managed servers where they exist, use external servers for third-party services (with credentials properly secured), and build custom servers **only** for the parts of your requirement that are genuinely proprietary. Building one giant custom server "to be safe," or ignoring an available managed/external server, is consistently the wrong answer pattern in this exam's scenario questions.
- Storing API keys/credentials for external MCP servers should go through **Databricks Secrets**, not embedded in agent code or prompts.

Common mistakes/traps

- Defaulting to "build it all custom" even when managed or external options exist for parts of the requirement — this is the single most common wrong-answer trap in MCP scenario questions.
- Embedding proprietary calculation logic directly "in the agent's prompt" instead of exposing it as a proper tool via a custom MCP server — logic belongs in tools, not prompt text.

Comparison with related concepts

	Managed MCP server	External MCP server	Custom MCP server
Hosted by	Databricks	Third party	You
Effort to set up	Lowest	Low-medium (configure credentials)	Highest (you build and maintain it)
Use when	Databricks offers one for your resource (e.g., UC)	A third-party service already has one	The logic/data is genuinely proprietary

Example exam questions (style)

- "An agent needs Unity Catalog access, Slack integration, and a proprietary calculation unique to the org. Which combination of MCP servers meets these requirements with least maintenance overhead?" → Managed UC server + external Slack server + one custom server for the proprietary logic only.

Topic 9: Persistent Agent Memory & State (Lakebase)

What is it?

Agent "memory" or "state" is information that needs to persist across turns of a conversation (short-term memory, e.g., conversation history) or across sessions entirely (long-term memory, e.g., user preferences, ongoing workflow state). This needs a durable datastore, separate from the vector index used for knowledge retrieval.

Why is it used?

Without persistent state, an agent forgets everything between messages or sessions — it can't resume a multi-step task, recall what a user said two turns ago, or maintain structured records (like "seller memory" in a marketplace agent) that need to be reliably read and updated as the conversation progresses.

How does it work?

Databricks offers **Lakebase**: a fully managed, serverless **Postgres**-compatible OLTP (transactional) database that lives inside the Databricks platform, co-located with your Lakehouse data.

- Provides **ACID guarantees** (Atomicity, Consistency, Isolation, Durability) — critical when an agent's memory must be updated reliably and consistently, not just "eventually" written
- Natively integrated with **Unity Catalog governance**

- Autoscales, including scale-to-zero when idle — good fit for "bursty" conversational traffic
- Supports frameworks like **LangGraph** (via `langgraph-checkpoint-postgres`) for checkpointing agent state, enabling resuming/forking conversations from any checkpoint

Important terminology

- **OLTP (Online Transaction Processing)** — a database workload pattern optimized for many small, fast, transactional reads/writes (as opposed to OLAP/analytics, which favors large scans and aggregation)
- **ACID guarantees** — the transactional reliability properties that make a database safe for concurrent, critical state updates
- **Checkpointing** — periodically saving an agent's/graph's state so a conversation can be resumed or "time-traveled" back to an earlier point

Databricks implementation

- Lakebase Postgres is available as a managed service; you create a Lakebase project/instance, then connect via standard Postgres drivers/SQLAlchemy
- For LangGraph-based agents, Lakebase can serve as the checkpointer backend, giving you thread-level conversation resumption
- Contrast with alternatives: a **vector store** is for knowledge retrieval, not transactional state; an **external NoSQL database** sits outside Unity Catalog governance; an **in-memory cache with periodic snapshotting** doesn't give durable ACID guarantees

Real-world example

An e-commerce sales assistant agent needs to: call tools to fetch order history/inventory/support tickets, AND maintain structured "seller memory" of recent interactions and open issues, AND provide ACID guarantees with Unity Catalog governance. Lakebase, used as the primary operational database for that seller memory, satisfies all three requirements simultaneously (transactional reliability + native UC governance), unlike a vector store (not transactional), an external NoSQL DB (not UC-governed), or an in-memory cache (not durable/ACID).

Simple example

```python
```

```python
# conceptual: LangGraph agent using Lakebase Postgres as a checkpointer
from langgraph.checkpoint.postgres import PostgresSaver

checkpointer = PostgresSaver.from_conn_string(lakebase_connection_string)
graph = build_agent_graph(checkpointer=checkpointer)

# thread_id lets the same conversation be resumed later
graph.invoke(input, config={"configurable": {"thread_id": "user_123_session_1"}})
```

Important exam points

- **Lakebase is a relatively new Databricks feature** — if you studied GenAI on Databricks before it existed, your instinct might default to "just use a vector store" or "use an external database" for agent state. Both fail requirements around ACID guarantees and/or Unity Catalog governance when those are explicitly stated.
- Match the requirement phrase "**ACID guarantees and governance in Unity Catalog**" directly to Lakebase — this is a strong, memorizable trigger phrase.

Common mistakes/traps

- Reaching for a vector store to hold structured, frequently-updated "memory" — vector stores are built for similarity search over relatively static embedded content, not transactional read/write state.
- Assuming an in-memory cache with periodic snapshotting is "good enough" for durability — snapshotting is eventual, not transactional; a crash between snapshots loses unsaved state.

Comparison with related concepts

	Vector Search	**Lakebase**
Purpose	Semantic similarity search over knowledge	Transactional storage of agent state/memory
Data pattern	Mostly write-once, read-many (embeddings)	Frequent reads AND writes (conversation state)
Guarantees	Not ACID-focused	ACID-compliant (Postgres)
Governance	Unity Catalog integrated	Unity Catalog integrated

Example exam questions (style)

- "An agent must maintain structured, frequently-updated 'memory' with ACID guarantees and Unity Catalog governance. Which Databricks service fits?" → Lakebase, used as the primary operational database for that memory.

Topic 10: Multi-Agent Systems with Genie Spaces

What is it?

Genie is Databricks' natural-language-to-SQL interface for querying Unity Catalog tables conversationally (part of AI/BI). A **Genie Space** is a configured, self-service analytics environment business users already interact with directly. Multi-agent systems can call an existing Genie Space **as a tool** rather than reimplementing SQL generation logic.

Why is it used?

If your organization already has business analysts using a Genie Space for self-service analytics over structured sales/operational data, an agent needing similar structured-data answers shouldn't reinvent SQL generation from scratch — it should reuse the already-built, already-trusted Genie Space.

How does it work?

Genie Spaces can be integrated into a Multi-Agent Supervisor (or a custom agent) as a callable tool via the **Genie Conversational API**, which lets the agent send a natural-language question to the Genie Space and receive structured query results back programmatically.

Important terminology

- **Genie Space** — a configured natural-language-to-SQL analytics environment over specific Unity Catalog tables
- **Genie Conversational API** — the programmatic interface that lets an agent (not just a human via UI) query a Genie Space and get results back

Databricks implementation

- Register the Genie Space as a **tool** in the agent's/supervisor's tool configuration, using the Conversational API
- This avoids: (a) writing custom SQL generation logic yourself, (b) connecting directly to a SQL warehouse and bypassing Genie's existing natural-language capability, and (c) embedding a Genie UI iframe (which is for human interactive use, not agent-to-agent programmatic calls)

Real-world example

A Multi-Agent Supervisor for a sales organization needs to answer business questions over structured sales data that a Genie Space already handles for self-service analysts. Rather than writing new SQL-generation logic, the supervisor calls the existing Genie Space as a tool via the Conversational API, retrieving query results programmatically to incorporate into its response.

Simple example

```
User → Supervisor: "What were our top 3 selling products in Q3 by region?"
Supervisor reasons: this is a structured-data query already served by our Genie Space
Supervisor → Genie Space (via Conversational API): "top 3 selling products in Q3 by region"
Genie Space → Supervisor: [structured result table]
Supervisor → User: formatted answer based on returned data
```

Important exam points

- The correct integration pattern is registering the Genie Space **as a tool via the Conversational API** — not writing custom SQL, not connecting directly to a warehouse, not embedding a UI iframe.
- This objective is explicitly named in Section 3 of the official exam guide: "Enable multi-agent systems to leverage Genie Spaces or conversational API to retrieve data."

Common mistakes/traps

- Bypassing an existing, working Genie Space to write new custom SQL-generation logic — unnecessary duplication of effort and loses whatever tuning/trust already exists in the Genie Space.
- Confusing "embed the Genie UI" (for humans) with "call Genie programmatically" (for agents) — these solve different problems.

Example exam questions (style)

- "A Multi-Agent Supervisor needs to answer questions by querying structured sales data that a Genie Space already serves for human analysts, without writing custom SQL. What should it do?" → Register the Genie Space as a tool using the Genie Conversational API.

Quick self-check before moving on

1. What's the difference between a chain and an agent, in terms of who controls the sequence of steps?

2. What's the difference between ReAct and Chain-of-Thought?
3. Name the four Agent Bricks types and one sentence on what each does.
4. When should you build a custom MCP server vs. use a managed or external one?
5. Why doesn't a vector store work well as agent "memory," and what does Lakebase provide instead?
6. How does an agent call an existing Genie Space programmatically?

Next: Study Material Part 3 — MLflow 3 for GenAI (Tracing, Prompt Registry, Scorers/Evaluation) & Model Serving / Foundation Model APIs (Sections 3, 4 & 6's MLflow- and serving-heavy objectives).